

Smart Lab Attendance System

Comprehensive Project Documentation, System Architecture, Database Schema, and Working Process

PROJECT REPORT & TECHNICAL SPECIFICATION • JULY 2026

1. Executive Summary & Overview

The **Smart Lab Attendance System** is an autonomous, multi-modal attendance verification and analytics platform designed specifically for academic computer laboratories and university departments. Built using Python, Streamlit, OpenCV, and SQLite, the system replaces manual roll-call registers with AI-driven face recognition, anti-spoofing liveness detection, and GPS geofencing.

The system enforces strict security protocols: students must be physically present within the college perimeter (verified via HTML5 Geolocation API and Geopy distance calculation) and pass a real-time eye-blink liveness challenge before attendance is marked. Additionally, administrators and faculty members can export structured attendance reports grouped by faculty member, subject, year, and semester as individual CSV files or bulk ZIP archives.

2. System Architecture & Tech Stack

The system is structured into five core technological layers:

Layer	Technology / Library	Function & Purpose
Frontend UI	Streamlit + Custom CSS (Inter)	Minimalist responsive web dashboard with role-based routing.
Computer Vision & AI	OpenCV + Face Recognition (Dlib)	HOG face detection, 68-point facial landmarks, and 128D feature encodings.
Geofencing & Security	Geopy + Streamlit JS Eval	HTML5 browser GPS extraction & geodesic distance calculation (40m radius lock).
Database Storage	SQLite3 (Embedded Relational DB)	Zero-config transactional storage for students, faculty, and attendance logs.
Audio & Voice	macOS Siri Speech Engine (say -v siri)	Real-time spoken audio feedback confirming attendance status.

3. Database Architecture & Schema (SQLite)

All application state and historical records are stored in a local SQLite database named **attendance.db**. On startup, the system automatically initializes three relational tables and migrates legacy CSV files if present.

A. Table Schemas

Table Name	Columns & Types	Description & Constraints
attendance	id INTEGER PRIMARY KEY AUTOINCREMENT roll_number TEXT NOT NULL date TEXT NOT NULL time TEXT NOT NULL lab TEXT NOT NULL	Logs every valid attendance entry marked by students via Face Recognition, QR Code, or Manual Bypass.
students	roll_number TEXT PRIMARY KEY registration_date TEXT face_encoding TEXT (JSON string) face_path TEXT	Stores registered student Roll Numbers, 128-dimensional face encoding vectors (serialized as JSON), and face image filepaths.
faculties	id INTEGER PRIMARY KEY AUTOINCREMENT name TEXT NOT NULL email TEXT NOT NULL department TEXT NOT NULL year TEXT semester TEXT	Stores faculty registrations mapped to specific subjects, academic years (1st, 2nd, 3rd Year), and semesters (1st to 6th Sem).

4. Working Process & End-to-End Workflow

- **Step 1: Geofencing Check** — When the user opens the application, JavaScript extracts browser latitude/longitude coordinates via HTML5 Geolocation API. Geopy computes distance to college location (15.2738° N, 76.3774° E). Access is blocked if the distance exceeds 40 meters.
- **Step 2: Role-Based Dashboard Routing** — Users interact with the app based on active session state:
 - *Student Mode*: Direct access to Face Scanner, QR Scanner, Face Registration, and Personal QR Code Generator.
 - *Faculty Mode*: View, search, filter daily lab attendance records, and delete records.
 - *Admin Mode*: Manage Faculty registry, Bulk/Per-Faculty CSV and ZIP downloads filtered by Subject and Semester, Attendance Analytics, and Manage Records.
- **Step 3: Student Enrollment** — When a student registers, their webcam/uploaded photo is analyzed by Dlib's facial recognition algorithm. A 128-dimensional floating-point encoding array is serialized into a JSON string and saved into the students SQLite table along with a JPEG snapshot saved under registered_faces/.jpg.
- **Step 4: Liveness Detection & Face Verification** — The camera feed captures frames continuously:
 1. Facial landmarks are extracted for left eye (landmarks 36-41) and right eye (landmarks 42-47).
 2. Eye Aspect Ratio (EAR) is calculated: $EAR = (|p1 - p5| + |p2 - p4|) / (2 * |p0 - p3|)$.
 3. The student is challenged to **blink their eyes**. A valid blink transition (open → closed → reopened) proves live presence, preventing spoofing via photos or smartphone screens.
- **Step 5: Attendance Rules & Recording** — Once liveness and face match are confirmed:
 - Attendance is inserted into the attendance SQLite table.
 - Daily attendance cap is enforced (max 1 session on Mon-Thu, max 2 sessions on Fri-Sat).
 - Spoken audio feedback is triggered using macOS Siri speech synthesis.
- **Step 6: Admin Attendance Export** — Admin selects date ranges, year, semester, or specific faculty. The system generates customized CSV reports per faculty member or a structured ZIP archive grouped by Year/Semester/Faculty_Subject_attendance.csv.

5. Detailed File-by-File Breakdown

File Name	Role & Responsibilities in Project
attend.py	Main Streamlit application (1,900+ lines of Python). Contains all business logic, SQLite helper functions (init_db, load_data, mark_attendance, enroll_student), OpenCV webcam loop, EAR blink liveness detection engine, and multi-tab UI rendering.
style.css	Custom CSS stylesheet delivering a sleek Minimalist dark theme (`#09090b` obsidian background), Inter font typography, custom Streamlit button styling, metric card containers, and modal popups.
attendance.db	SQLite relational database file storing relational tables for attendance, students, and faculties.
registered_faces/	Directory storing JPEG image snapshots of registered student faces, named as .jpg.
qrcores/ & qrcode_saver.py	Directory and utility script for generating and storing downloadable PNG QR code images for student roll numbers.
requirements.txt	Dependency manifest file specifying required packages: streamlit, face_recognition, opencv-python, pandas, geopy, streamlit_js_eval, qrcode, Pillow.

6. Servers & Database Architecture

A. Database Engine (SQLite)

The project utilizes **SQLite3**, an embedded SQL database engine. SQLite requires zero server configuration, operates directly on local file storage (attendance.db), and provides full ACID compliance with high read/write performance suitable for local academic lab deployments.

B. Server & Deployment Options

The application can be hosted across multiple server environments:

- **Local Server (Current):** Launched via streamlit run attend.py serving on http://localhost:8501 (or 8503) accessible within campus LAN.
- **Cloud VPS / EC2 Deployment:** Can be hosted on Ubuntu Linux (AWS EC2 / DigitalOcean) behind an Nginx Reverse Proxy with SSL (HTTPS) to allow remote access.
- **Enterprise Scaling (PostgreSQL / MySQL Migration):** For multi-building or multi-campus deployments, SQLite database calls in attend.py can be swapped to PostgreSQL / MySQL with central server connectivity.

7. Advantages & Disadvantages

Advantages (✓)	Disadvantages & Limitations (X)
• Anti-Spoofing Security: Real-time Eye Aspect Ratio (EAR) blink detection prevents photo/screen proxy attacks. • Geofenced Attendance: GPS radius lock ensures students must be inside college premise. • Granular Reports: Download attendance filtered per faculty, per subject, and per semester as CSV or ZIP. • Zero DB Server Setup: SQLite embedded database operates seamlessly without external database servers. • Multi-Modal Flexibility: Supports Face Recognition, QR Code Scanning, and Manual Faculty Bypass.	• Webcam Hardware Requirement: Requires a functional camera on user devices. • Browser Geolocation Permission: Users must grant location permissions in browser settings. • Indoor GPS Signal Limits: Deep basement computer labs may experience weak GPS signals. • Single Server File Locking: SQLite is optimal for single-server setups; multi-server clusters require PostgreSQL.